Naive Bayes and Gaussian Naive Bayes with Grid Search and Random Search

In this assignment, we are required to implement the Naive Bayes algorithm on the datasets used in previous assignments. To accomplish this, we first need to preprocess the data. Then, using the following code snippet, we implement Naive Bayes, a probability-based classification algorithm, and evaluate its performance.

We begin by applying the algorithm to the IMDB dataset used in Assignment 2. The following code is added to implement the model:

```
count_vectorizer = CountVectorizer()

train_text = [" ".join([reverse_word_index.get(i - 3, "?") for i in doc]) for doc in train_data]
test_text = [" ".join([reverse_word_index.get(i - 3, "?") for i in doc]) for doc in test_data]

nb_pipeline = Pipeline([
   ('vect', count_vectorizer),
   ('clf', MultinomialNB())
])
nb_pipeline.fit(train_text, train_labels)
nb_preds = nb_pipeline.predict(test_text)

print("Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, nb_preds))
print("Classification Report:\n", classification_report(test_labels, nb_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels, nb_preds))
```

The output is shown below. For each class, it prints the Confusion Matrix, the overall accuracy, and the evaluation metrics (Recall, Precision, and F1-score).

We observe that the accuracy achieved by this method is 0.82368, which is lower than the accuracy of the model in Assignment 2. However, it is still a reasonably good result.

The Confusion Matrix can be interpreted as follows:

- TP (True Positive): 9,731 samples were correctly predicted as belonging to the Positive class (class 1).
- TN (True Negative): 10,861 samples were correctly predicted as belonging to the Negative class (class 0).
- FP (False Positive): 1,639 samples were incorrectly predicted as belonging to class 1.
- FN (False Negative): 2,769 samples were incorrectly predicted as belonging to class 0.

Using the Confusion Matrix and the corresponding formulas, Recall, Precision, and F1-score are calculated and presented below.

```
Naive Bayes Classifier:
Accuracy: 0.82368
Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.87      0.83     12500
           1       0.86      0.78      0.82     12500

    accuracy                           0.82     25000
   macro avg       0.83      0.82      0.82     25000
weighted avg       0.83      0.82      0.82     25000

Confusion Matrix:
 [[10861  1639]
 [ 2769  9731]]
```

In this model, we also applied Smoothing using the following code in an attempt to improve the model's accuracy. However, it did not have any noticeable effect.

```
alpha = 1.0

pipeline = Pipeline([
    ('vect', CountVectorizer()),
    ('clf', MultinomialNB(alpha=alpha))
])
```

Next, we implement the Gaussian Naive Bayes model:

```
count_vectorizer = CountVectorizer()

train_text = count_vectorizer.fit_transform([" ".join([reverse_word_index.get(i - 3, "?") for i in doc]) for doc in
train_data]).toarray()
test_text = count_vectorizer.transform([" ".join([reverse_word_index.get(i - 3, "?") for i in doc]) for doc in
test_data]).toarray()

gnb_pipeline = Pipeline([
    ('clf', GaussianNB())
```

```
])
gnb_pipeline.fit(train_text, train_labels)
gnb_preds = gnb_pipeline.predict(test_text)

print("Gaussian Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, gnb_preds))
print("Classification Report:\n", classification_report(test_labels, gnb_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels, gnb_preds))
```

The results are shown below.

```
Gaussian Naive Bayes Classifier:
Accuracy: 0.66264
Classification Report:
              precision    recall  f1-score   support

           0       0.62      0.86      0.72     12500
           1       0.77      0.47      0.58     12500

    accuracy                           0.66     25000
   macro avg       0.69      0.66      0.65     25000
weighted avg       0.69      0.66      0.65     25000

Confusion Matrix:
 [[10739  1761]
 [ 6673  5827]]
```

We observe that the Gaussian model achieves lower accuracy than the previous model. The difference between these two models lies in the assumptions on which they are based.

The standard Naive Bayes model assumes that all features are independent of one another. In contrast, Gaussian Naive Bayes assumes that the features are continuous and follow a Gaussian (Normal) distribution, which we studied in the Probability and Statistics course.

Since the data in the IMDB dataset is text-based and its features do not follow a Gaussian distribution, the Gaussian model achieves lower accuracy and is not considered a suitable algorithm for predicting these data. In general, this model does not perform well on datasets whose features do not follow a Gaussian distribution. In contrast, standard Naive Bayes is more suitable for text-based data, which typically have discrete features, and therefore achieves better performance in this case.

Applying the Models to the First Assignment's Datasets

Now, we move on to the datasets from Assignment 1.

First Dataset: persian_dataset = load_dataset("SeyedAli/Persian-Text-Sentiment")
We implement Naive Bayes by running the following code:

```
 persian_dataset = load_dataset("SeyedAli/Persian-Text-Sentiment")

texts = [doc['text'] for doc in persian_dataset['train']]
labels = [doc['label'] for doc in persian_dataset['train']]

train_texts, test_texts, train_labels, test_labels =
train_test_split(texts, labels, test_size=0.2, random_state=42)

count_vectorizer = CountVectorizer()

nb_pipeline = Pipeline([
    ('vect', count_vectorizer),
    ('clf', MultinomialNB())
])
nb_pipeline.fit(train_texts, train_labels)
nb_preds = nb_pipeline.predict(test_texts)

print("Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, nb_preds))
print("Classification Report:\n", classification_report(test_labels,
nb_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels, nb_preds))
```

The output is shown below. For each class, it calculates and prints the final accuracy, the Confusion Matrix, and the evaluation metrics (Recall, Precision, and F1-score) based on the Confusion Matrix.

```
Naive Bayes Classifier:
Accuracy: 0.8467460388505953
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.91      0.86      5649
           1       0.89      0.78      0.83      5522

    accuracy                           0.85     11171
   macro avg       0.85      0.85      0.85     11171
weighted avg       0.85      0.85      0.85     11171

Confusion Matrix:
 [[5139  510]
 [1202 4320]]
```

We observe that the accuracy achieved by this method is 0.846746, which is lower than the accuracy obtained for this dataset in the previous assignment.

The Confusion Matrix can be interpreted as follows:

- TP (True Positive): 4,320 samples were correctly predicted as belonging to the Positive class (class 1).
- TN (True Negative): 5,139 samples were correctly predicted as belonging to the Negative class (class 0).
- FP (False Positive): 510 samples were incorrectly predicted as belonging to class 1.
- FN (False Negative): 1,202 samples were incorrectly predicted as belonging to class 0.

Next, we implement Gaussian Naive Bayes:

```
persian_dataset = load_dataset("SeyedAli/Persian-Text-Sentiment")

texts = [doc['text'] for doc in persian_dataset['train']]
labels = [doc['label'] for doc in persian_dataset['train']]

train_texts, test_texts, train_labels, test_labels =
train_test_split(texts, labels, test_size=0.2, random_state=42)

count_vectorizer = CountVectorizer()

train_features = count_vectorizer.fit_transform(train_texts).toarray()
test_features = count_vectorizer.transform(test_texts).toarray()

gnb = GaussianNB()
gnb.fit(train_features, train_labels)
gnb_preds = gnb.predict(test_features)

print("Gaussian Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, gnb_preds))
print("Classification Report:\n", classification_report(test_labels,
gnb_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels, gnb_preds))
```

```
Gaussian Naive Bayes Classifier:
Accuracy: 0.5690627517679706
Classification Report:
              precision    recall  f1-score   support

           0       0.54      0.92      0.68      5649
           1       0.71      0.21      0.33      5522

    accuracy                           0.57     11171
   macro avg       0.63      0.57      0.51     11171
weighted avg       0.63      0.57      0.51     11171
```

```
Confusion Matrix:
 [[5178  471]
 [4343 1179]]
```

We observe that, for this dataset as well, the Gaussian model achieves lower accuracy than the standard Naive Bayes model. This is due to the nature of the dataset and the underlying assumptions of the two models, for the same reasons discussed when applying them to the previous dataset.

Now, we test the models on the `Persian_news_dataset`:

```
dataset = load_dataset("saied/persian_news_dataset", split="train",
streaming=True)

# Extracting features and labels
texts = []
labels = []
document_count = 1000
counter = 0
skip_counter = 0

for doc in dataset:
    if len(doc['title']) == 0 or len(doc['category']) == 0:
        skip_counter += 1
        if skip_counter % 10000 == 0:
            print(f'{skip_counter} skipped')
        continue
    texts.append(doc['text'])
    labels.append(doc['category'])
    counter += 1
    if counter == document_count:
        break

train_texts, test_texts, train_labels, test_labels =
train_test_split(texts, labels, test_size=0.2, random_state=42)

count_vectorizer = CountVectorizer()

train_features = count_vectorizer.fit_transform(train_texts)
test_features = count_vectorizer.transform(test_texts)

nb_classifier = MultinomialNB()
nb_classifier.fit(train_features, train_labels)
```

```
nb_preds = nb_classifier.predict(test_features)

print("Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, nb_preds))
print("Classification Report:\n", classification_report(test_labels,
nb_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels, nb_preds))
```

As in the previous experiments, we also implement Gaussian Naive Bayes for this dataset. The results show that the Gaussian model achieves lower accuracy than the standard Naive Bayes model.
As mentioned earlier, all three datasets used in this assignment are text-based datasets. Gaussian Naive Bayes assumes that continuous features follow a Gaussian distribution. However, text data often do not satisfy this assumption, which can result in poor model performance.
Furthermore, based on my research, text datasets typically have high-dimensional feature spaces and sparse representations, such as Bag of Words and TF-IDF. These types of features may not be modeled effectively by a Gaussian distribution, which can lead to lower accuracy.

Implementing Naive Bayes with Grid Search

Grid Search is a hyperparameter optimization method in which we define a set of possible values for each parameter. The algorithm then evaluates all possible combinations of these values and selects the best-performing combination.

We applied this method to the first dataset and observed that adding Grid Search improved the model's accuracy compared to the initial result.

```
train_text = [" ".join([reverse_word_index.get(i - 3, "?") for i in doc])
for doc in train_data]
test_text = [" ".join([reverse_word_index.get(i - 3, "?") for i in doc])
for doc in test_data]

count_vectorizer = CountVectorizer()

pipeline = Pipeline([
    ('vect', count_vectorizer),
    ('clf', MultinomialNB())
])

param_grid = {
    'vect__max_features': [1000, 5000, 10000],
    'vect__ngram_range': [(1, 1), (1, 2)]
}

grid_search = GridSearchCV(pipeline, param_grid, cv=5, n_jobs=-1)
```

```
grid_search.fit(train_text, train_labels)

print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)

grid_search_preds = grid_search.best_estimator_.predict(test_text)

print("Grid Search + Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, grid_search_preds))
print("Classification Report:\n", classification_report(test_labels,
grid_search_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels,
grid_search_preds))
```

In the code below, `vect__max_features` represents the maximum number of words (features) to be considered. We assign three different values to this parameter. The parameter `vect__ngram_range` specifies the range of n-gram sizes. As we learned in class, for example, a 1-gram (unigram) considers each word individually.

All possible combinations of these parameter values are evaluated, and the best combination is selected, resulting in the following accuracy:

```
Best Parameters: {'vect__max_features': 10000, 'vect__ngram_range': (1,
2)}
Best Score: 0.8470000000000001
Grid Search + Naive Bayes Classifier:
Accuracy: 0.84368
Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.86      0.85     12500
           1       0.85      0.83      0.84     12500

    accuracy                           0.84     25000
   macro avg       0.84      0.84      0.84     25000
weighted avg       0.84      0.84      0.84     25000

Confusion Matrix:
 [[10688  1812]
 [ 2096 10404]]
```

As mentioned above, we can observe that using Grid Search improves the model's accuracy.

Implementing Naive Bayes with Random Search

Next, we use Random Search to compare the results.

```
train_text = [" ".join([reverse_word_index.get(i - 3, "?") for i in doc])
for doc in train_data]
test_text = [" ".join([reverse_word_index.get(i - 3, "?") for i in doc])
for doc in test_data]
param_dist = {
    'vect__max_features': randint(1000, 10000),
    'vect__ngram_range': [(1, 1), (1, 2)]
}

random_search = RandomizedSearchCV(pipeline,
param_distributions=param_dist, n_iter=10, cv=5, n_jobs=-1)
random_search.fit(train_text, train_labels)

print("Best Parameters (Random Search):", random_search.best_params_)
print("Best Score (Random Search):", random_search.best_score_)

random_search_preds = random_search.best_estimator_.predict(test_text)

print("Random Search + Naive Bayes Classifier:")
print("Accuracy:", accuracy_score(test_labels, random_search_preds))
print("Classification Report:\n", classification_report(test_labels,
random_search_preds))
print("Confusion Matrix:\n", confusion_matrix(test_labels,
random_search_preds))
```

Random Search works by randomly selecting values for `vect__max_features` from the specified range and then evaluating the selected combinations to find the best one.

The output is shown below:

```
Best Parameters (Random Search): {'vect__max_features': 7321,
'vect__ngram_range': (1, 2)}

Best Score (Random Search): 0.8412
Random Search + Naive Bayes Classifier:
Accuracy: 0.83952
Classification Report:
               precision    recall  f1-score   support

           0       0.83      0.85      0.84     12500
           1       0.85      0.83      0.84     12500

    accuracy                           0.84     25000
   macro avg       0.84      0.84      0.84     25000
weighted avg       0.84      0.84      0.84     25000

Confusion Matrix:
```

```
[[10622  1878]
 [ 2134 10366]]
```

We observe that its accuracy is higher than that of the initial model but lower than the accuracy achieved using Grid Search.

Both Grid Search and Random Search are hyperparameter optimization methods. Based on these results, we can conclude that these methods can be useful for improving our model's accuracy.